



# Практическое задание

## Цель

1. Закрепить **структуру HTTP-запроса и ответа** (стартовая строка, заголовки, тело).
2. Понять **маршрутизацию** (как путь + метод выбирают обработчик).
3. Научиться **валидировать контракт**: методы, статус-коды, Content-Type, Authorization, ETag/кэш.

## Задание (шаги)

1. **Запустите проект**
  - Убедитесь, что сервер стартует на `http://localhost:5080`.
2. **Найдите в консоли “REQUEST START LINE”**
  - Для каждого клиентского запроса выпишите:
    - Метод
    - URI (путь + query)
    - Версию протокола (из `ctx.Request.Protocol`)
3. **Проверьте соответствие метода и смысла**
  - Объясните, почему:
    - `/users` с `GET` безопасный
    - `/users` с `POST` не идемпотентный
    - `/users/{id}` с `PUT` идемпотентный
    - `/users/{id}` с `DELETE` идемпотентный
4. **Валидируйте заголовки запроса**
  - Для `POST/PUT/PATCH` :
    - Проверьте, что `Content-Type = application/json`
    - Проверьте, что есть `Authorization: Bearer demo-token-123`
  - Для `GET` :
    - Проверьте роль `Accept`
5. **Валидируйте коды состояния и структуру ответа**
  - Сопоставьте эндпоинты и ожидаемые статусы:
    - `POST /users` без токена → `401`
    - `POST /users` с токеном → `201` + `Location`
    - `GET /users/{id}` → `200` или `304` при ETag

- `DELETE` → `204`
- `/boom` → `500` (Problem Details, `application/problem+json`)
- Для каждого ответа объясните:
  - что означает код
  - какие заголовки важны (например, `Etag`, `Cache-Control`, `Allow`)

## 6. Проверьте stateless на практике

- Удалите у клиента установку `Authorization` и убедись, что сервер не “помнит” тебя между запросами.
- Добавьте обратно `Authorization` — запрос снова работает.
- Объясните: *почему это stateless и где “состояние” передаётся явно.*

## 7. Маршрутизация

- Объясните на примере:
  - почему `GET /users` попадает в один обработчик
  - а `HEAD /users` — в другой
  - и почему `GET /users/123` — вообще третий маршрут

# Подсказки по ключевым частям: что за что отвечает

- **Middleware с логом стартовой строки:** показывает основу HTTP — *что именно отправили* (метод + URI + версия).
- **FakeAuth.IsAuthorized:** демонстрирует stateless-авторизацию через заголовок `Authorization`.
- **Content-Type проверки:** демонстрируют контракт тела запроса. Неверный формат → `415 Unsupported Media Type`.
- **Etag + If-None-Match:** демонстрируют кэширование HTTP-ответов и `304 Not Modified`.
- **MapMethods OPTIONS/HEAD:** демонстрируют методы, которые часто игнорируют новички, но они важны (особенно OPTIONS в браузере).
- **Маршруты /users и /users/{id}:** демонстрируют разницу “коллекция ресурсов” vs “конкретный ресурс”.

## Что проверить перед отправкой (чек-лист)

- ☐ Я могу объяснить **клиент–сервер**: кто инициирует запрос и почему сервер “не звонит первым”.
- ☐ Я могу показать **строгую модель запрос–ответ**: один запрос → один ответ.

- ☐ Я понимаю **stateless** и могу показать, где в запросе передаётся состояние (Authorization / cookie / query / headers).
- ☐ Я различаю **Accept** и **Content-Type** и знаю, кто за что отвечает.
- ☐ Я могу сопоставить **методы** и **свойства** (безопасность/идемпотентность).
- ☐ Я умею объяснить **маршрутизацию**: метод + путь → обработчик.
- ☐ Я знаю, из чего состоит **ответ**: статусная строка + заголовки + тело.
- ☐ Я понимаю смысл **кодов 2xx/3xx/4xx/5xx** на примерах из кода.

## Советы по улучшению работы (как усилить проект)

### 1. Добавьте RequestId-трассировку в ответы

- Сейчас мы читаем `X-Request-Id` из запроса. Можно возвращать его в ответ, чтобы связывать логи клиента и сервера.

### 2. Сделайте более строгую валидацию тела

- Например, проверять email по regex и возвращать `422 Unprocessable Entity`.

### 3. Покажите cookies как механизм “состояния поверх stateless”

- Добавь эндпоинт `/login`, который возвращает `Set-Cookie`, и затем проверяй cookie в последующих запросах.

### 4. Добавьте контент-неготиацию

- Если `Accept: application/xml` — возвращать XML (даже примитивно), иначе JSON.

### 5. Расширьте обработчик ошибок (Problem Details)

- Примените единый формат ошибок ко всем 4xx/5xx (например, RFC 7807), не только к `/boom`.